

Assignment 08 – Data-Driven Automation (DDT)

1. Module Name

02_Unit_Test_Frameworks

2. Experiment Title

Data-Driven Automation (DDT)

3. Problem Statement

Build a Selenium Python login automation script that reads multiple test cases from an external source. The script should execute both correct and incorrect username/password combinations and verify the expected successful login or validation error for each test case.

4. Objective

The objective of this experiment is to implement Data-Driven Testing using an external CSV file. The same Selenium login workflow is reused for multiple sets of test data, reducing duplication and making the test data easy to modify independently of the automation code.

5. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Google Chrome and ChromeDriver
- VS Code
- CSV test data source
- Python csv module
- Selenium locators
- WebDriverWait and Expected Conditions
- Loops
- Assertions
- Data-Driven Testing (DDT)

6. Source Code / Implementation Steps

1. Create an external CSV file containing test case ID, username, password and expected result.
2. Add both valid and invalid login combinations to the CSV file.
3. Read the CSV file using Python's csv module.
4. Create one reusable function to execute a login test case.
5. Open the SauceDemo login page for every test case.
6. Enter the username and password from the current CSV record.
7. Click the Login button.
8. For the valid case, verify that the URL contains /inventory.html.
9. For invalid cases, wait for the validation error and verify that it is displayed and contains text.
10. Repeat the same workflow for all test records using a loop.
11. Display the total number of executed and passed test cases.

7. Output / Labelled Screenshots

The following screenshots are captured during execution and stored in **Source_Code/Screenshots/**:

- Screenshot 1 – TC01 Successful Login
- Screenshot 2 – TC02 Validation Error
- Screenshot 3 – TC03 Validation Error
- Screenshot 4 – TC04 Validation Error
- Screenshot 5 – Successful Data-Driven Execution in Terminal

8. Result

The login automation successfully read all test cases from the external CSV file and executed them one by one. The valid credentials produced a successful login, while the invalid credential combinations produced the expected validation errors. Each test case was verified using Selenium assertions.

9. Observation

Data-Driven Testing allows the same automation logic to be reused with different input values. Keeping test data in a separate CSV file makes the automation easier to maintain and allows additional test cases to be added without changing the main Selenium workflow.

10. Conclusion

This experiment provided practical experience in implementing Data-Driven Automation using Selenium Python. I learned how to store test cases externally, read them programmatically, execute multiple combinations through a loop, and verify the expected result for each case. The assignment was completed successfully.